KS Assistant: A Simple General-Purpose AI Agent for Software Engineering

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large language models can generate code and call tools with remarkable fluency, yet deploying them as practical software engineering assistants still exposes stubborn gaps: finite context windows, a single mistake that can derail entire sessions, agents that get stuck in dead ends, AI slop, and generated changes that are difficult to review or revert.

We present **KS Assistant**, a general-purpose assistant and integrated development environment (IDE) built on top of the **KS Agent Framework**, a simple AI agent framework of roughly 1,900 lines of code. The framework addresses these gaps through a robust system prompt and a five-layer agent hierarchy in which each layer adds exactly one concern: budget-tracked ReAct execution, automatic continuation across sub-sessions via summarization, coding and browser tools with parallel sub-agents, persistent multi-turn chat with history recall, and git worktree isolation so every task runs on its own branch. Both KS Assistant and the KS Agent Framework are grounded in disciplined software engineering practice; these principles are encoded directly into the agent's system prompt, enabling KS Assistant to write code that is simple, elegant, maintainable, and bug-free.

We implemented KS Assistant as a free, open-source Visual Studio Code extension that runs locally, is effective for long-horizon tasks, and supports browser automation, multimodal input, and Docker containers. We deliberately prioritize output quality over speed: giving a frontier model adequate time to validate its own output—running linters, type checkers, and tests—dramatically reduces the low-quality code that plagues faster but less thorough agents. The entire system was built using itself over four months, providing a continuous stress test in which any bug was patched immediately after its manifestation. On Terminal Bench 2.0, KS Assistant achieves a 62.2% overall pass rate with Claude Opus 4.6, comparing favorably to Claude Code (58%) and Cursor Composer 2 (61.7%). These results are achieved without benchmark-specific prompt tuning or model fine-tuning.

1 Introduction

Modern large language models (LLMs), such as Claude Opus 4.7 [Anthropic, 2026], GPT 5.5 [OpenAI, 2026], and Gemini 3.1 [Google DeepMind, 2026], have demonstrated a remarkable ability to generate code, reason about software architecture, and use developer tools [Chen et al., 2021, Rozière et al., 2023]. A growing body of work has explored how to harness these capabilities for autonomous software engineering, from single-session agents that resolve GitHub issues [Yang et al., 2024, Wang et al., 2024] to industrial products marketed as AI software and general assistants [GitHub, 2021, Cursor, 2024, Cognition Labs, 2024, Anthropic, 2025, OpenAI, 2025a, OpenClaw AI, 2025]. Yet using an LLM as a practical software engineering assistant still exposes several stubborn gaps: context windows are finite, a single mistake can derail an entire session, agents get stuck in dead

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

ends, models generate AI slop, and generated changes are difficult to review or revert once applied to a live codebase.

We propose the *KS Agent Framework*, a simple AI agent framework containing around 1,900 lines of code for the core agent implementation. The name "KS" reflects the *Keep Simple* design philosophy: each layer is small, each concern is isolated, and the overall system avoids unnecessary abstraction. We address the above-mentioned gaps through a robust system prompt (Section 4 and Appendix A) and a five-layer agent hierarchy in which each layer solves exactly one concern:

1. **KS Agent** — budget-tracked ReAct [Yao et al., 2023b] loop with native function calling.
2. **Relentless Agent** — automatic summarization and continuation across sub-sessions.
3. **Tool Agent** — coding tools, browser automation, and parallel sub-agent execution.
4. **Chat Agent** — persistent multi-turn chat sessions with history recall.
5. **Worktree Agent** — git worktree isolation so every task runs on its own branch.

To demonstrate the power of the KS Agent Framework, we implemented KS Assistant as a Visual Studio Code extension that runs locally. It has full browser support (using open-source Chromium and Playwright), multimodal support, Docker container support, and a mobile/web app. KS Assistant is completely free and open-source; all one needs is a model API key from a major LLM provider. The open-source repository link is withheld to respect double-blind review.

KS Assistant has been built using itself. The entire codebase—the KS Agent framework, the agent layers, the VS Code extension, and the system prompt—was developed by KS Assistant operating on its own repository. This self-hosting discipline provides a continuous-integration-style stress test: if the agent introduces a bug that impairs its ability to function, the developers immediately ask the agent to fix it by analyzing the trajectory and code. The simplicity of the framework enabled the agent to implement features confidently without bugs and AI slop. The five core agent classes are remarkably compact: the KS Agent comprises 413 lines, the Relentless Agent 321 lines, the Tool Agent 322 lines, the Chat Agent 125 lines, and the Worktree Agent 704 lines—a total of roughly 1,885 lines of code (excluding empty lines and comments).

We deliberately prioritize output quality over speed. Using a weaker or cheaper model often forces the developer to discard the agent's work and retry, ultimately increasing the total cost. Conversely, giving a frontier model adequate time to validate its own output—running linters, type checkers, and tests before declaring success—dramatically reduces slop. We expect token costs and inference latencies to continue to fall [Gao et al., 2025], making this quality-first posture increasingly practical.

We evaluate on Terminal Bench 2.0 and achieve a 62.2% overall pass rate using Claude Opus 4.6, comparing favorably to Claude Code (58%) and Cursor Composer 2 (61.7%) [Cursor Research, 2026] on the same benchmark (Section 3). These results are achieved without benchmark-specific prompt tuning or model fine-tuning. We show that a simple agent framework, without sophisticated agent technologies such as trajectory compaction and asynchronous multi-agent orchestration, is sufficient to build a competitive software engineering assistant. By building KS Assistant using itself and matching or beating both the Cursor and Claude Code agents, we demonstrate that time-tested software engineering techniques and principles are invaluable for building reliable, practical agent systems.

Outline. Section 2 presents the five-layer agent architecture. Section 3 reports evaluation results on Terminal Bench 2.0. Section 4 describes the two most distinctive aspects of the system prompt: the execution mindset and the web research protocol (the full prompt is detailed in Appendix A). Section 5 covers unique features of the VS Code extension. Section 6 discusses related work, and Section 7 concludes.

2 Agent Architecture

The KS Agent Framework was originally built to rapidly prototype and experiment with prompt optimization techniques [Agrawal et al., 2026] and evolutionary algorithms for algorithmic optimization [Novikov et al., 2025, Algorithmic Superintelligence, 2025]. The emphasis on simplicity enabled coding agents to write simple, bug-free code. No prompt optimization techniques were used when creating the system prompt for KS Assistant; it was hand-tuned based on long-term experience

with the agent and its behavior. The framework uses five agent layers combining composition and inheritance. Each layer delegates upward for concerns it does not own.

2.1 KS Agent

The KS Agent is the innermost execution unit implementing a standard ReAct loop [Yao et al., 2023b].

```python
from ks.core.ks_agent import KSAgent

def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

agent = KSAgent(name="Math Buddy")
result = agent.run(
    model_name="gemini-2.5-flash",
    prompt_template="Calculate: {question}",
    arguments={"question": "What is 15% of 847?"},
    tools=[calculate]
)
print(result)  # 127.05
```

Listing 1: A complete KS agent with a single tool.

Native function calling. Tools are registered as ordinary Python callables. The agent builds an OpenAI-compatible tool schema once at setup time and caches it. A special `finish` tool signals task completion and returns the result.

Step, token, and budget tracking. At every step, the agent extracts input and output token counts from the API response, computes dollar cost using a per-model pricing table, and updates both local and global budget counters protected by a class-level lock. Three limits are checked before each step: per-agent budget, global budget, and maximum step count.

Error resilience. The agent retries transient API errors (rate limits, server errors) up to a configurable threshold. Non-retryable errors (authentication failures, permission denials) are raised immediately.

Non-agentic mode. When tools are not needed, the agent runs a single generation without the ReAct loop, useful for summarization sub-tasks.

The KS Agent is stateless across runs: each call resets the conversation, token counters, and tool registry, making it safe to reuse a single instance for multiple sequential tasks.

2.2 Relentless Agent

The Relentless Agent wraps a KS Agent in a continuation loop. Its core contribution is executing tasks that exceed a single context window by breaking them into sub-sessions.

Rather than investing in context-compaction techniques, we adopt a simple continuation protocol: when a sub-session exhausts its context window or step budget, the agent produces a *structured summary* of every action taken so far—chronologically ordered, with explanations and relevant code snippets—and a fresh sub-session resumes from that summary. This approach is related to Reflexion [Shinn et al., 2023], which feeds verbal self-critiques back into subsequent trials, but uses a Reflexion-like technique to *continue* a task rather than retry it. We found that a naïve instruction to "summarize the current context" produced poor continuations; requiring a *step-by-step chronological account with code snippets* dramatically improved coherence across sub-sessions.

Continuation protocol. The `finish` tool accepts three fields: a success flag, a continue flag, and a summary. When `is_continue=True`, the Relentless Agent starts a new sub-session with a fresh context window containing all prior attempt summaries.

Forced continuation on failure. If a sub-session raises an exception (e.g., the step limit is hit before calling finish), the Relentless Agent saves the full trajectory to a temporary file, spawns a separate summarizer agent to produce a concise summary, and uses that summary as the progress text for the next sub-session. This ensures that even crashed sessions contribute useful context.

2.3 Tool Agent

The Tool Agent adds the tools that make the system useful for software development and general-purpose automation.

Coding tools. Four core tools: a shell command executor with streaming output, a file reader, a precise string-based file editor, and a file writer. The shell executor supports configurable timeouts, streams output in real time, and respects a stop event for user cancellation.

Browser automation. A web-use tool provides programmatic browser control: navigating to URLs, reading page accessibility trees, clicking elements, typing text, pressing keys, scrolling, and taking screenshots. It uses the open-source Chromium browser via the Playwright library.

Parallel sub-agents. An optional parallel execution tool spawns independent Tool Agent instances in a thread pool. Each sub-agent gets its own LLM context and tool set, useful for embarrassingly parallel tasks such as summarizing multiple files or researching independent topics.

User interaction. An ask-user-question tool pauses execution and requests clarification from the user.

Docker isolation. When a Docker image is specified, coding tools are replaced with Docker-aware variants that execute commands inside a container.

2.4 Chat Agent

The Chat Agent adds multi-turn conversation persistence.

Chat sessions. Each task is assigned to a chat session identified by a stable chat ID. Tasks and results are persisted to a local database. New tasks within the same session include prior tasks and results as numbered context entries, allowing the LLM to reference earlier work.

Bounded chat context. The agent caps in-context history entries at $K = 10$. When the cap is exceeded, it preserves the first two entries (which establish the user's intent) and the most recent entries, dropping the middle entries least likely to be referenced.

Session management. Three operations are supported: starting a new chat, resuming by task description, and resuming by explicit chat ID.

2.5 Worktree Agent

The Worktree Agent is the outermost layer. Its defining feature is git-worktree isolation.

Branch-per-task. When a task starts, the agent creates a new git branch and corresponding worktree directory. All agent modifications happen inside the worktree; the user's main working tree remains untouched.

Dirty-state preservation. Uncommitted changes in the main working tree are copied into the worktree with a baseline commit. During merge, cherry-pick from the baseline replays only the agent's changes.

Concurrency safety. A per-repository file lock serializes checkout, stash, merge, and pop operations. Thread-local storage isolates per-task state.

Crash recovery. All worktree state is stored in git itself (branch names, git config entries) rather than sidecar files. On process restart, the agent queries git and reconstructs instance attributes, enabling seamless recovery.

Graceful fallback. If the working directory is not inside a git repository, has no commits, or has a detached HEAD, the agent falls back to direct execution without worktree isolation.

3 Evaluation on Terminal Bench 2.0

We evaluate on Terminal Bench 2.0,[1] a benchmark comprising 89 diverse terminal-based programming tasks, ranging from building legacy compilers and configuring servers to solving cryptanalysis

[1] `https://www.tbench.ai/`

challenges and training machine-learning models. Each task runs in an isolated Docker container; a separate verifier automatically judges the result. We use the Harbor[2] framework to orchestrate execution and Claude Opus 4.6 as the underlying LLM. We do not modify the general system prompt or inject benchmark-specific instructions. Evaluation was carried out on a 2025 MacBook Air 15" with an M4 processor and 24 GB RAM.

3.1 Setup

We run 5 independent trials per task. The agent is a thin Harbor adapter that installs and invokes the KS Assistant CLI inside each container. We hard-skip 9 tasks verified to be infeasible for Opus 4.6 across 6+ prior attempts (e.g., CompCert compilation, Windows 3.11 GUI installation, video OCR) to save time and token cost. Skipped tasks still count as failures.

3.2 Aggregate Results

Table 1 summarizes the aggregate statistics.

Table 1: Terminal Bench 2.0 aggregate results (89 tasks, 5 trials each, Claude Opus 4.6).

Metric	**Value**
Total tasks	89
Overall pass rate	62.2% (277/445)
pass@any (at least 1/5 passes)	78.7% (70/89)
pass@all (all 5 pass)	43.8% (39/89)
Always-fail tasks	19
Always-pass tasks	39
Mixed-result tasks	31
Median cost per trial	$0.45
Mean cost per trial	$0.90
Median duration per trial	202 s
Mean duration per trial	446 s

The 62.2% overall pass rate compares favorably to other agents using the same underlying model: at the time of writing, Claude Code (also Opus 4.6) scores approximately 58% on the Terminal Bench 2.0 leaderboard, and Cursor's Composer 2—a custom fine-tuned model trained with large-scale reinforcement learning [Cursor Research, 2026]—achieves 61.7%. Our result suggests that the layered architecture and the structured system prompt contribute meaningfully beyond what the base model provides.

3.3 Task-Level Breakdown

Consistently solved tasks (39 of 89). These include cryptanalysis (FEAL differential), game-playing (chess best move), git operations (leak recovery), server configuration (gRPC key-value store, PyPI server, NGINX logging), data processing (resharding), formal verification (Coq `plus_comm`), ML inference (HuggingFace model serving, LLM batching scheduler), and system emulation (QEMU startup). The breadth of this set demonstrates that the agent's generality is not limited to a single domain.

Consistently failed tasks (19 of 89). The failures cluster into three categories: (1) tasks requiring graphical or multimedia capabilities unavailable in the container, (2) tasks demanding very long or resource-intensive builds that exceed time or memory limits, and (3) tasks with niche domain-specific requirements that the model struggles to satisfy.

Mixed-result tasks (31 of 89). Tasks such as `write-compressor` (3/5), `crack-7z-hash` (4/5), and `feal-linear-cryptanalysis` (4/5) succeed in most trials but occasionally fail due to non-determinism in the model's reasoning or timing-sensitive environment interactions.

Leaderboard context. KS Assistant does not score as high as some coding agents on the Terminal Bench 2.0 leaderboard. However, we note that recent analysis has found widespread cheating

[2] `https://github.com/harbor-framework/harbor`

on popular agent benchmarks, including Terminal Bench 2.0: the top three submissions commit harness-level cheating (e.g., leaking verifier code or answer keys into the agent's environment), and task-level cheating (e.g., Googling answers, mining git history, hardcoding test outputs) affects 28+ submissions across 9 benchmarks [Stein et al., 2026a,b]. Separately, an automated benchmark audit found 45 confirmed hacking solutions across 13 widely used benchmarks exhibiting process-isolation failures, answer leakage, and weak test assertions that allow perfect scores without solving a single problem [Wang et al., 2026].

4 The System Prompt

The system prompt is a structured document that governs the agent's behavior across all tasks. It is not a generic instruction to "be helpful" but rather a precise specification of engineering discipline. We present the two most distinctive aspects here; the complete prompt is detailed in Appendix A.

4.1 Execution Mindset

The prompt opens with two directives that set the tone for the entire session:

```
# FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.
# BE RELENTLESS. BE CALM. BE RIGOROUS. BE ACCURATE. CHECK FACTS. NO AI SLOP.
```

Each directive addresses a specific failure mode observed in LLM-based agents:

"FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL." LLMs have a tendency to drift: they comment on the difficulty of a problem, explore tangential concerns, or ask clarifying questions that the user has already answered. This directive anchors the model on the task at hand and discourages meta-commentary that consumes tokens without making progress.

"BE RELENTLESS." When an agent encounters an error—a failing test, a type violation, a command that returns unexpected output—the default LLM behavior is to apologize, summarize the failure, and ask the user what to do next. This directive instructs the model to treat errors as obstacles to overcome, not as reasons to stop.

"BE CALM." The complement of relentlessness. An overly aggressive agent may thrash between approaches without deliberation. This directive encourages the model to analyze errors methodically before reacting.

"BE RIGOROUS." This instructs the model to follow the verification and testing disciplines encoded later in the prompt, rather than taking shortcuts when a solution *looks* right.

"BE ACCURATE." LLMs frequently generate plausible-but-wrong code, file paths, or command-line flags. This directive raises the model's threshold for asserting facts, encouraging it to verify claims against the actual file system or documentation rather than relying on parametric memory.

"CHECK FACTS." A more specific version of "be accurate," targeting information the agent collects via web tools.

"NO AI SLOP." "AI slop" refers to the low-quality, generic, hedging text that LLMs produce when they lack confidence: filler phrases like "certainly!", unnecessary caveats, and boilerplate explanations. This directive encourages the model to be concise and substantive.

These two sentences occupy only two lines, yet they establish a behavioral contract that permeates every subsequent interaction. We observe empirically that removing any single directive degrades the agent's behavior along the corresponding dimension (e.g., removing "BE RELENTLESS" causes the agent to give up after the first error more frequently).

4.2 Web Research Protocol

When the agent needs external knowledge, the prompt prescribes a structured research workflow rather than allowing ad-hoc browsing:

```
## Use web tools when you need to:
```

```

- When you need to collect knowledge from the internet,
  visit **AT LEAST 30 WEBSITES** (use a counter to keep
  track of the number of websites you visited) and collect
  information necessary for the task without much thinking
  in a new file PWD/tmp/information-{unique_id}.md. Then go
  over the information in
  PWD/tmp/information-{unique_id}.md and think deeply about
  how to complete the task at hand.
- If you need to log in to a website while browsing for
  information, you MUST ask the user to help you with the
  login.
```

The rationale is a two-phase separation between *collection* and *synthesis*. LLMs tend to anchor on the first few results they encounter, biasing their solutions toward a narrow slice of the design space. By forcing the agent to accumulate a broad set of information into a file *before* reasoning about it, the protocol counteracts anchoring bias and encourages the model to consider diverse approaches.

The "at least 30 websites" threshold is deliberately aggressive: it ensures the agent does not shortcut the research phase after two or three hits, and the resulting information file serves as an auditable artifact of what the agent considered. The explicit "use a counter" instruction addresses an empirically observed failure mode in which the model claims to have "visited many sites" after only a handful of fetches; a concrete counter forces the model to verify the actual number visited before declaring the collection phase complete.

This two-phase collect-then-synthesize discipline is also applied to local file browsing (Appendix A.9): the agent writes a structured summary of each file's relevant information into a temporary markdown file, externalizing its understanding into a compact artifact that persists across context boundaries. The instruction to collect "without much thinking" is deliberate: during the collection phase, the agent extracts and records facts rather than analyzes. Analysis happens in the second phase, when the agent reads its own summary and reasons about the collected information as a whole.

5 VS Code Extension: Unique Features

We release our system as a VS Code extension and a web app. While the underlying agent architecture already differs from existing AI coding assistants, the extension introduces several features that, to our knowledge, are not present in competing assistants—including GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], Windsurf [Codeium, 2024], Devin [Cognition Labs, 2024], and Aider [Gauthier, 2023].

Cross-session self-improvement. The extension maintains a `USER_PREFS.md` file that the agent reads at the start of each task and *updates* at the end. When the agent discovers a new preference or project invariant during task execution, it writes it to the file. The agent also resolves conflicts: if a new preference contradicts an existing one, the old entry is removed. Over time, the file accumulates a curated set of project-specific knowledge, making the agent progressively more effective without any manual configuration. This mechanism is detailed in Appendix A.7.

Real-time budget accountability. The extension displays real-time cost tracking in the sidebar: input tokens, output tokens, cache hits, dollar cost, and elapsed time are updated at every agent step. Both per-task and global budget ceilings are enforced.

Integrated browser automation. The extension includes a browser automation tool that allows the agent to navigate URLs, read accessibility trees, click elements, type text, press keys, scroll, and take screenshots—all controlled programmatically from within a VS Code task.

6 Related Work

Code-specialized language models. Code Llama [Rozière et al., 2023] fine-tunes Llama 2 for code generation. StarCoder [Li et al., 2023] trains on permissively licensed code. DeepSeek-Coder [Guo et al., 2024] trains on a 2-trillion-token corpus. Frontier models such as Claude Opus 4.7 [Anthropic, 2026], GPT 5.5 [OpenAI, 2026], Kimi K2.5 [Kimi Team, 2026], and GLM-5.1 [Z.ai, 2026] target agentic software engineering. Our system is model-agnostic and benefits from advances in code-specialized pre-training without architectural changes.

Code generation agents. SWE-Agent [Yang et al., 2024] and OpenHands [Wang et al., 2024] provide LLM-based agents for resolving software engineering tasks. Agentless [Xia et al., 2024] shows that a two-phase localize-then-repair pipeline can achieve competitive results. Devin [Cognition Labs, 2024], Claude Code [Anthropic, 2025], Codex [OpenAI, 2025a], and Aider [Gauthier, 2023] offer various approaches to autonomous coding. Cursor released Composer 2 [Cursor Research, 2026], a fine-tuned coding model trained with large-scale reinforcement learning. Our Relentless Agent layer addresses the single-session limitation common to most of these systems, while our worktree isolation provides stronger safety guarantees.

ReAct, reasoning, and planning. ReAct [Yao et al., 2023b] interleaves reasoning and action. Chain-of-thought prompting [Wei et al., 2022] elicits step-by-step reasoning. Tree of Thoughts [Yao et al., 2023a] generalizes to search over reasoning paths. Reflexion [Shinn et al., 2023] introduces verbal reinforcement learning. Our continuation protocol is conceptually related to Reflexion, but uses summaries to continue tasks rather than retry them.

Multi-agent systems. ChatDev [Qian et al., 2024] and MetaGPT [Hong et al., 2024] model software development as conversations between role-playing agents. AutoGen [Wu et al., 2023] provides a general multi-agent framework. We use a single agent with broad tool access and optional parallel sub-agents, prioritizing practical utility over role-playing fidelity.

Agent frameworks. LangChain [LangChain, 2022], DSPy [Khattab et al., 2024], CrewAI [CrewAI, Inc., 2024], smolagents [Roucher et al., 2025], the OpenAI Agents SDK [OpenAI, 2025b], and Google's ADK [Google, 2025] provide general-purpose agent infrastructure. Our architecture is purpose-built for software engineering, with each layer addressing a specific practical concern.

Benchmarks and evaluation. SWE-bench [Jimenez et al., 2024], HumanEval [Chen et al., 2021], LiveCodeBench [Jain et al., 2024], and SWE-bench Pro [Deng et al., 2025] evaluate coding agents at various scales. Prathifkumar et al. [2025] raises concerns about benchmark contamination. We evaluate on Terminal Bench 2.0, which tests diverse terminal-based tasks in isolated Docker containers.

Agentic software engineering. Hassan et al. [2025] articulate the foundational pillars of agentic software engineering. Li et al. [2025] surveys the landscape of autonomous coding agents. Wang et al. [2025] surveys AI agentic programming techniques. Chatlatanagulchai et al. [2025] studies agent context files. Mohsenimofidi et al. [2025] investigates context engineering for AI agents. Our system prompt and per-repository override mechanisms are instances of the context-engineering patterns these works advocate.

Self-improvement and community prompts. Robeyns et al. [2025] demonstrates a self-improving coding agent. Our self-improvement loop captures a lighter-weight form of this idea via accumulated preferences. Gao et al. [2025] applies test-time compute scaling to software engineering. Get Shit Done (GSD) [TÂCHES, 2025] is a meta-prompting system for coding agents that addresses context rot through phased planning documents. Parts of our system prompt were inspired by this work.

7 Conclusion

We have shown that a simple layered, single-concern architecture can address the practical challenges of deploying LLM agents for real-world software development. On Terminal Bench 2.0, a benchmark of 89 diverse terminal-based tasks evaluated across 5 trials each, our system achieves a 62.2% overall pass rate (277/445)—outperforming Claude Code (58%) and Cursor Composer 2 (61.7%) on the same benchmark with the same underlying model (Claude Opus 4.6). These results are achieved without benchmark-specific optimizations, fine-tuning, or reinforcement learning: the entire agent is roughly 1,900 lines of straightforward Python.

We complement the architecture with a system prompt that encodes engineering discipline directly into the agent's behavior: read before writing, test before fixing, plan before executing, verify before finishing. These are not novel insights—they are the practices of careful software engineering, translated into instructions that an LLM can follow. The evaluation suggests that giving a frontier model the time and tools to validate its own output matters more than model-level customization.

In summary, we showed that a simple agent framework, without sophisticated agent technologies such as trajectory compaction and asynchronous multi-agent orchestration, is sufficient to build a competitive software engineering assistant. By building the system using itself and matching

or beating both the Cursor and Claude Code agents, we demonstrated that time-tested software engineering techniques and principles are invaluable for building reliable, practical agent systems.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. Oral. arXiv preprint arXiv:2507.19457.

Algorithmic Superintelligence. OpenEvolve: Open-source implementation of AlphaEvolve. `https://github.com/algorithmicsuperintelligence/openevolve`, 2025.

Anthropic. Claude Code: Anthropic's agentic coding system. `https://www.anthropic.com/product/claude-code`, 2025.

Anthropic. Introducing Claude Opus 4.7. `https://www.anthropic.com/news/claude-opus-4-7`, 2026.

Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Codeium. Windsurf: The AI-powered IDE. `https://windsurf.com`, 2024.

Cognition Labs. Devin: The first AI software engineer. `https://devin.ai`, 2024.

CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. `https://github.com/crewAIInc/crewAI`, 2024.

Cursor. Cursor: The AI-first code editor. `https://cursor.sh`, 2024.

Cursor Research. Composer 2 technical report. *arXiv preprint arXiv:2603.24477*, 2026.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

Pengfei Gao, Zhao Tian, Xiangxin Meng, and Trae Research Team. Trae agent: An LLM-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.

Paul Gauthier. Aider: AI pair programming in your terminal. `https://github.com/paul-gauthier/aider`, 2023.

GitHub. GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`, 2021.

Google. Agent Development Kit (ADK): An open-source framework for building AI agents. `https://google.github.io/adk-docs/`, 2025.

Google DeepMind. Gemini 3.1 Pro. `https://deepmind.google/models/gemini/pro/`, 2026.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yun Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216*, 2025.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.

Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

LangChain. LangChain: Build context-aware reasoning applications. `https://github.com/langchain-ai/langchain`, 2022.

Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of AI teammates in software engineering (SE 3.0): How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*, 2025.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: May the source be with you! *Transactions on Machine Learning Research (TMLR)*, 2023.

Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. Context engineering for AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI. Introducing Codex: A cloud-based software engineering agent. `https://openai.com/index/introducing-codex/`, 2025a.

OpenAI. OpenAI Agents SDK: A lightweight, powerful framework for multi-agent workflows. `https://github.com/openai/openai-agents-python`, 2025b.

OpenAI. Introducing GPT-5.5. `https://openai.com/index/introducing-gpt-5-5/`, 2026.

OpenClaw AI. OpenClaw: Personal AI assistant. `https://openclaw.ai`, 2025.

Thanosan Prathifkumar, Noble Saji Mathews, and Meiyappan Nagappan. Does SWE-Bench-Verified test agent ability or model memory? *arXiv preprint arXiv:2512.10218*, 2025.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smolagents: A smol library to build great agentic systems. `https://github.com/huggingface/smolagents`, 2025.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Finding widespread cheating on popular agent benchmarks. Blog post, `https://debugml.github.io/cheating-agents/`, 2026a.

Adam Stein, Davis Brown, Hamed Hassani, Mayur Naik, and Eric Wong. Detecting safety violations across many agent traces. *arXiv preprint arXiv:2604.11806*, 2026b.

TÂCHES. Get Shit Done: A light-weight meta-prompting, context engineering and spec-driven development system for AI coding agents. `https://github.com/gsd-build/get-shit-done`, 2025. Initial commit December 2025.

Hao Wang, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. We scored 100% on AI benchmarks without solving a single problem. Blog post, `https://moogician.github.io/blog/2026/trustworthy-benchmarks/`, 2026.

Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. AI agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhi Li, Hao Peng, and Heng Ji. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.

Z.ai. GLM-5.1: Towards long-horizon tasks. Technical blog, `https://z.ai/blog/glm-5.1`, 2026.

A Full System Prompt Details

This appendix presents the remaining sections of the system prompt not discussed in the main body. The execution mindset and web research protocol are covered in Sections 4.1 and 4.2, respectively.

A.1 Tool Rules

Tool usage rules are explicit and mechanical:

```
# Rules

- PWD denotes your current working directory.
- Write() for new files. Edit() for small changes.
- Run Bash commands synchronously using the
  ‘timeout_seconds‘ parameter. Use 300s (default) for
  commands. If a command times out, retry with a higher
  timeout. Only for commands expected to take more than
  10 minutes, run them in the background, redirect output
  to a file, and poll periodically.
- Use go_to_url() for browser tool.
- **The user cannot see intermediate chat. Show whatever
  the user asks in the summary of the ’finish’ tool
  call.**
- READ large files in chunks.
- Create temporary files in PWD/tmp. Cleanup temporary
  files after the task is done.
- Use ULTRA thinking ALWAYS.
- **If you are running out of context length or steps,
```

```
  DO NOT rush to complete the task urgently, but continue
  the task by calling 'finish' with 'is_continue' set
  to true**
```

Each rule addresses a specific failure mode:

"Write() for new files. Edit() for small changes." Without this distinction, the model may use `Write()` to overwrite an existing file with a slightly modified version, losing content it forgot to include.

Bash timeout guidance. LLMs frequently launch shell commands without considering their runtime. The instruction to use 300 seconds as the default, retry with higher timeouts, and run long-running commands in the background provides a mechanical protocol that handles common cases.

"The user cannot see intermediate chat." This forces the model to put all user-facing information into the `finish` summary, preventing situations where the model "tells" the user something in an intermediate message and then assumes it was seen.

"READ large files in chunks." Reading a 10,000-line file in a single tool call consumes a large fraction of the context window. Chunked reading preserves capacity for the rest of the task.

"Create temporary files in PWD/tmp." Centralizing temporary files in a known directory makes cleanup predictable and prevents project-tree pollution.

"Use ULTRA thinking ALWAYS." This activates the model's extended reasoning mode, which is particularly valuable for complex, multi-step tasks.

"If you are running out of context length or steps…" When the context window is nearly full, LLMs exhibit a "rush to finish" behavior, skipping verification steps. This instruction redirects that urgency into the continuation protocol (Section 2.2).

A.2 Pre-flight Checks

Before modifying any file, the agent is instructed to read it first:

```
## Pre-flight Checks

- Read every file you will modify before changing it.
- If the task depends on existing architecture or behavior,
  read the relevant source files first.
- If the task references files, commands, or config that do
  not exist, stop and ask or report instead of guessing.
- **When fixing a bug, an issue, or a race, write tests to
  confirm them. Then fix them.**
```

"Read every file you will modify before changing it." The most common source of agent-introduced bugs is modifying a file based on an incorrect assumption about its current contents. By requiring a fresh read immediately before any edit, the instruction ensures the model operates on the file's current state.

"When fixing a bug… write tests to confirm them. Then fix them." This mandates test-first discipline: a test that reproduces the bug provides concrete verification that the fix is correct. Writing the test forces the model to understand the bug precisely before attempting a fix.

A.3 Code Style Guidelines

The prompt encodes a minimalist code philosophy:

```
## Code Style Guidelines

- Write simple, clean, and readable code with minimal
  indirection.
- Organize the code in multiple files based on the code's
  functionality.
- Avoid unnecessary object attributes, local variables, and
  config variables.
- Avoid tight coupling among files and modules.
- Avoid object/struct attribute redirections.
```

```
- DO NOT USE CLOSURES.
- No redundant abstractions or duplicate code.
- Public methods MUST have full documentation.
- Understand the root cause of an issue or bug, and patch
  the root cause instead of an ad hoc superficial fix.
- Before you write code, wait and think if the code is
  simple, elegant, general, and minimal.
- Once you finish the task, DO NOT write documentation
  unless the task specifically requires it.
```

Each guideline addresses an anti-pattern commonly exhibited by LLM-generated code. For example, "DO NOT USE CLOSURES" steers the model toward explicit data structures (plain functions with arguments, classes with attributes) that are easier to test and reason about. "Before you write code, wait and think" is a metacognitive instruction that asks the model to pause and evaluate its plan, spending more inference-time compute on design.

A.4 Deep Work Rules

```
## Deep Work Rules

- When the task says "align", "match", or "make
  consistent", read the target to determine the exact
  target state before editing.
- Use concrete values, not indirections.
- For multi-part work, list the concrete planned changes
  before executing them.
- Every meaningful change should have a concrete
  verification method (test, grep, CLI command).
```

These rules address failures where the model interprets instructions loosely and makes changes that are directionally correct but concretely wrong.

A.5 Planning for Complex Tasks

```
## Planning for Complex Tasks

For tasks involving 3+ files, cross-module changes, or
architectural work:

1. List the files that need to change and why.
1. State the exact intended change in each file.
1. Identify dependencies and execution order.
1. State how each change will be verified.

For simple single-file tasks, skip formal planning and
execute directly.
```

The complexity threshold—three or more files—avoids the overhead of planning trivial changes while ensuring comprehensive planning for tasks with cross-module dependencies.

A.6 Testing Instructions

```
## Testing Instructions

- Run lint and typecheckers and fix any lint and typecheck
  errors.
- You MUST achieve 100% branch coverage.
- Tests MUST NOT use mocks, patches, fakes, or any form of
  test doubles.
- You MUST write integration tests or end-to-end tests.
- Each test should be independent and verify actual behavior.
- **Do NOT run all tests after modifications. Only run the
  impacted tests.**
- To confirm a race condition, add sleep statements before
  racing statements with delays less than 0.1s.
```

The most opinionated rule is the no-mocks requirement. Mock tests verify that code calls certain methods in a certain order—they test the implementation, not the behavior. A test suite built on mocks can pass with flying colors while the system is fundamentally broken. Integration tests that exercise actual behavior provide genuine confidence that the system works.

A.7 Self-Improvement Loop

The agent maintains a preferences file that captures user invariants discovered during task execution:

```
## Self-Improvement Loop

- Read the instructions in PWD/USER_PREFS.md at the start
  of each task.
- Then update PWD/USER_PREFS.md to capture the user
  preferences and invariants by analyzing the task.
- You MUST carefully and thoroughly get rid of the user
  preferences and invariants that conflict with the newly
  added ones.
```

This mechanism allows the agent to accumulate project knowledge over time without requiring the user to repeat themselves. The conflict-resolution instruction mandates that the agent actively resolves contradictions when updating the file.

A.8 Pre-Finish Verification

```
## Pre-Finish Verification

Before calling finish(success=True, ...), you MUST:

1. Re-read every file you modified and verify the changes
   are correct.
1. Run the required checks (lint, typecheck, tests) and fix
   any failures.
1. Explicitly check each user requirement against what was
   delivered.
1. If any check fails, continue working instead of
   finishing.
1. If you have retried the same fix 3 times without
   progress, step back, rethink the approach from scratch,
   and try a different strategy.
```

The three-retry threshold forces the model to break out of repetitive loops by abandoning the current approach and reconsidering the problem from first principles.

A.9 File Browsing Protocol

```
## Browsing files for a task

- When you need to read files for a task, collect
  information, including code snippets necessary for the
  task, without much thinking, in a new file
  PWD/tmp/file-information-{unique_id}.md. Then go over
  the information in
  PWD/tmp/file-information-{unique_id}.md, think deeply
  on how to complete the task at hand.
```

This instruction applies the same two-phase collect-then-synthesize discipline used for web research (Section 4.2) to the local file system. By writing a structured summary of each file's relevant information into a temporary markdown file, the agent externalizes its understanding into a compact artifact that is typically much smaller than the raw source files, freeing up context window capacity for subsequent reasoning and editing.

A.10 Desktop Application Control

```
## Launch desktop apps

- Use screenshots, keyboard, and mouse to control a desktop
  app.
- Do not launch VS Code or its extensions.
```

The prohibition on launching VS Code prevents a recursive loop: since the agent runs inside a VS Code extension, launching another instance could corrupt the host session or create deadlocks.

B Painless Software Engineering

A central claim of our system is that natural-language interaction can replace manual code inspection and ad hoc scripting for understanding and evolving nontrivial subsystems. We illustrate this with a real development session in which the worktree merge workflow (Section 2.5) was first understood and then redesigned entirely through conversational prompts.

Step 1: Understanding the existing workflow. The developer asks the agent to explain the current post-task git lifecycle. The agent reads the source code and returns a structured summary of the four-phase lifecycle.

Step 2: Simplifying the workflow. The developer decides the three-way choice (auto-merge, manual merge, discard) is unnecessarily complex and issues a redesign request in natural language. The agent modifies six files across Python and TypeScript, removes the manual-merge method, simplifies the UI to two buttons, updates tests, and verifies all 28 worktree tests pass.

Step 3: Investigating unexpected state. After testing, the developer notices files appearing in the Source Control panel and asks why. The agent traces the execution path and discovers the implementation deliberately unstages changes for user review.

Step 4: Directing a design change. The developer decides the extra review step is unnecessary and says "Fix it" in one sentence. The agent replaces the unstage call with a conditional commit, adds an edge-case guard, updates one test, and verifies all 104 worktree tests pass.

The developer never opens a source file, never writes a line of code, and never runs a test manually. This style of development becomes possible because of the agent hierarchy: the Worktree Agent isolates changes on a branch, the Chat Agent preserves context across tasks, and the Relentless Agent continues when the context window is exhausted.